

Vehicle Parking App – V1 (MAD1T22025)

AUTHOR

Name: Praphull Kumar Shahi

Roll Number: 23f3001743

Email: 23f3001743@ds.study.iitm.ac.in

About me :

I am a 2nd year student of BS Degree in data science at IIT Madras . A passionate student exploring full-stack development and backend design.

DESCRIPTION

This project is a multi-user web application for managing 4-wheeler parking lots, spots, and reservations. It supports two roles: Admin (superuser, no registration required) and User (registers, logs in, reserves, and vacates spots). The app enables efficient parking management and provides dashboards and summary charts for both roles.

TECHNOLOGY

- Flask (App + Routing)
- SQLite + SQLAlchemy (Database and ORM)
- HTML + CSS + Bootstrap (Frontend Styling)
- Jinja2 (Templating)
- Chart.js (Charts for summary)
- Flask-Login (User authentication)
- Werkzeug (password hashing)

DB SCHEMA DESIGN

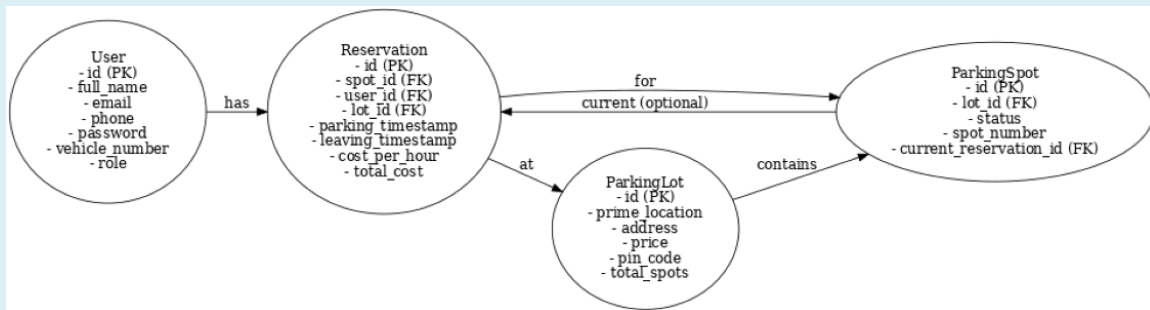
Entities:

- **User:** id, full_name, email, phone, password, vehicle_number, role
- **ParkingLot:** id, prime_location, address, price, pin_code, total_spots

- **ParkingSpot:** id, lot_id (FK), status, spot_number, current_reservation_id
- **Reservation:** id, spot_id (FK), user_id (FK), lot_id (FK), parking_timestamp, leaving_timestamp, cost_per_hour, total_cost

Relationships:

- User (1) — (M) Reservation
- ParkingLot (1) — (M) ParkingSpot
- ParkingLot (1) — (M) Reservation
- ParkingSpot (1) — (M) Reservation
- ParkingSpot (1) — (0..1) Reservation (current reservation)



API ENDPOINTS

- /search_spot (search by spot or lot ID, returns HTML)
- /user/search-by-pincode (search by pincode, returns HTML)
- (No public JSON APIs; all endpoints are web routes)

I ARCHITECTURE AND FEATURES

1 Architecture

- **MVC Pattern:**
 - All route logic and controller functions are in app.py.
 - Database models are in models.py using SQLAlchemy.
 - HTML templates for different user roles are in the templates/ folder, rendered with Jinja2.

- **Static files (CSS, JS) are in the static/ folder.**

Key Features

- **Admin:**
 - Create, edit, and delete parking lots (only if all spots are empty)
 - View all users and see parking spot status
 - View summary charts for spot usage and revenue
- **User:**
 - Register/login
 - Reserve a spot (first available in a lot; cannot select a specific spot)
 - Can only have one active reservation at a time; must vacate before booking another
 - Vacate spot and see total cost based on parking duration
 - View reservation history and dashboard with charts
- **Other Features:**
 - Search by pincode and by spot/lot ID
 - Responsive, user-friendly UI with clear navigation
 - All data stored in SQLite, created via models

Folder Structure:

```
text
/Parking_App/
  app.py
  models.py
  /templates/
  /static/
```

PRESENTATION VIDEO LINK

https://drive.google.com/file/d/1WzV-apNzWtj_59L9bAyQzuBdyzwfpbGU/view?usp=sharing

AI/LLM USAGE

For this project, I used AI tools (such as ChatGPT) for guidance, code review, and code correction. I also used AI to help me understand and implement Chart.js for dashboard charts, and to verify that my code followed best practices. All business logic, templates, and the final implementation were written and tested by me.

The AI tools were used only for support, learning, and to ensure my code was robust and error-free. All core logic and design decisions are my own.